Metodología de la programación

PL2, parte 1: Manejo de excepciones y errores en los programas

Estos ejercicios están basados en los ejercicios propuestos en la Universidad Tecnológica de Nanyang en Singapur.

1.1 Introducción

Una excepción es un **evento anormal** que surge durante la ejecución del programa y perturba el flujo normal del programa. Las anomalías ocurren cuando el programa está en ejecución. Por ejemplo, podrías esperar que el usuario introduzca un entero por teclado, pero recibes una cadena de texto; o un error inesperado de E/S (Entrada/Salida) aparece en tiempo de ejecución. Lo que realmente importa es "¿qué sucede después de que ocurre una anomalía?" En otras palabras, "cómo las situaciones anormales son manejadas por tu programa."

Si estas excepciones no se manejan adecuadamente, el programa termina abruptamente y puede causar graves consecuencias.

Por ejemplo, ***las conexiones de red, conexiones a bases de datos y archivos*** pueden permanecer abiertos; registros de bases de datos y archivos pueden quedar en un estado inconsistente. Java tiene un mecanismo integrado para manejar errores en tiempo de ejecución, referido como **manejo de excepciones**. Esto es para asegurar que puedas escribir programas **robustos** para aplicaciones críticas.

Lenguajes de programación más antiguos como C tienen algunas desventajas en el manejo de excepciones. Por ejemplo, supongamos que el programador desea abrir un archivo para procesarlo:

1. Los programadores no son informados de las condiciones excepcionales. Por ejemplo, el archivo que se va a abrir no necesariamente existe. Por lo tanto, el programador no escribió códigos para probar si el archivo existe antes de abrirlo.
2. Supongamos que el programador está al tanto de las condiciones excepcionales, podría decidir terminar la lógica principal primero, y escribir los códigos de manejo de excepciones más tarde – este "más tarde", desafortunadamente, usualmente nunca sucede. En otras palabras, no estás obligado a escribir los códigos de manejo de excepciones junto con la lógica principal.
3. Supongamos que el programador decidió escribir los códigos de manejo de excepciones, los códigos de manejo de excepciones **se entrelazan** con la lógica principal en muchos enunciados if-else. Esto hace que la lógica principal sea difícil de seguir y todo el programa difícil de leer.

Por ejemplo:

```
if (file exists) {
    open file;
    while (there is more records to be processed) {
        if (no IO errors) {
            process the file record
        } else {
```

```
            handle the errors
        }
    }
    if (file is opened) close the file;
} else {
    report the file does not exist;
}
```

Java supera estas desventajas al integrar el manejo de excepciones en el lenguaje en lugar de dejarlo a discreción de los programadores: 1. Serás informado sobre las condiciones excepcionales que pueden surgir al llamar a un método - Las excepciones están declaradas en la firma del método. 2. Estás obligado a manejar las excepciones mientras escribes la lógica principal y no puedes dejarlas como una reflexión posterior - Tu programa no puede compilarse sin los códigos de manejo de excepciones. 3. Los códigos de manejo de excepciones están separados de la lógica principal - A través de la construcción **try-catch-finally**. Veamos estos tres puntos con más detalle.

Punto 1: Las Excepciones Deben Ser Declaradas

Como ejemplo, supongamos que quieres usar un **java.util.Scanner** para leer una entrada formateada desde un archivo en disco.

La ***firma*** del constructor de **Scanner** con un argumento **File** se da de la siguiente manera:

```
public Scanner(File source) throws FileNotFoundException;
```

La firma del método informa a los programadores que puede surgir una condición excepcional de "archivo no encontrado". Al declarar las excepciones en la firma del método, se hace consciente a los programadores de las condiciones excepcionales al usar el método.

Punto 2: Las Excepciones Deben Ser Manejadas

Si un método declara una excepción en su firma, no puedes usar este método sin manejar la excepción - no puedes compilar el programa.

Ejemplo 1: El programa no manejó la excepción declarada, resultando en un error de compilación.

```
import java.util.Scanner;
import java.io.File;
public class ScannerFromFile {
    public static void main(String[] args) {
        Scanner in = new Scanner(new File("test.in"));
        // do something ...
    }
}
```

```
ScannerFromFile.java:5: unreported exception java.io.FileNotFoundException; must be caught or declared to be thrown
        Scanner in = new Scanner(new File("test.in"));
                                 ^
```

Para usar un método que declara una excepción en su firma, **DEBES**:

1. proporcionar códigos de manejo de excepciones en una construcción de "try-catch" o "try-catch-finally", ***o***

2. no manejar la excepción en el método actual, pero declarar que la excepción se propague hacia arriba en la pila de llamadas para que el método de nivel superior la maneje.

Ejemplo 2: Atrapar la excepción mediante una construcción de "try-catch" (o "try-catch-finally").

```
import java.util.Scanner;

import java.io.File;
import java.io.FileNotFoundException;
public class ScannerFromFileWithCatch {

    public static void main(String[] args) {

        try {
            Scanner in = new Scanner(new File("test.in"));
            // do something if no exception ...
            // you main logic here in the try-block
        } catch (FileNotFoundException ex) { // error handling separated from the main logic
            ex.printStackTrace();             // print the stack trace
        }
    }

}
```

Si el archivo no se puede encontrar, la excepción es capturada en el bloque catch. En este ejemplo, el manejador de errores simplemente imprime el ****rastreo de pila****, que proporciona información útil para la depuración. En algunas situaciones, puede que necesites realizar algunas operaciones de limpieza, o abrir otro archivo en su lugar. Ten en cuenta que la lógica principal en el bloque try está separada de los códigos de manejo de errores en el bloque catch.

Ejemplo 3: Decides no manejar la excepción en el método actual, pero lanzar la excepción hacia arriba en la pila de llamadas para que el método de nivel superior la maneje.

```
import java.util.Scanner;

import java.io.File;
import java.io.FileNotFoundException;
public class ScannerFromFileWithThrow {
    public static void main(String[] args) throws FileNotFoundException {
                                          // to be handled by next higher-level method
        Scanner in = new Scanner(new File("test.in"));
                                         // this method may throw FileNotFoundException
        // main logic here ...
    }

}
```

En este ejemplo, decides no manejar la **FileNotFoundException** lanzada por el método **Scanner(File)** (con **try-catch**). En su lugar, el método que llama a **Scanner(File)** - el método **main()** - declara en su firma "**throws FileNotFoundException**", lo que significa que esta excepción se lanzará hacia arriba en la pila de llamadas, para que el método de nivel superior la maneje. En este caso, el método de nivel superior a **main()** es la JVM, que simplemente termina el programa e imprime el rastreo de pila.

Punto 3: La lógica principal está separada de los códigos de manejo de excepciones

Como se muestra en el Ejemplo 2, la lógica principal está contenida en el bloque try, mientras que los códigos de manejo de excepciones se mantienen en el(los) bloque(s) catch separados de la lógica principal. Esto mejora grandemente la legibilidad del programa.

Por ejemplo, un programa de Java para el procesamiento de archivos podría ser como sigue:

```
try {
    // Main logic here
    open file;
```

```
    process file;

    ......
} catch (FileNotFoundException ex) {   // Exception handlers below
    // Exception handler for "file not found"
} catch (IOException ex) {
    // Exception handler for "IO errors"
} finally {
   close file;      // always try to close the file
}
```

1.2 Explorando la pila de llamadas

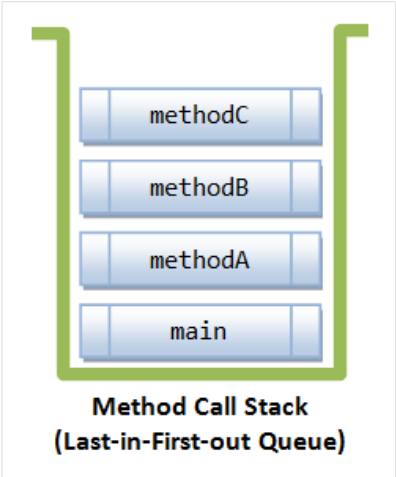
Una aplicación involucra muchos niveles de llamadas a métodos, lo cual es gestionado por lo que se llama una **pila de llamadas a métodos**. Una **pila** es una cola de **último en entrar, primero en salir**. En el siguiente ejemplo, el método **main()** invoca a **methodA();** **methodA()** llama a **methodB();** **methodB()** llama a **methodC().**

```
public class MethodCallStackDemo {
    public static void main(String[] args) {
        System.out.println("Enter main()");
        methodA();
        System.out.println("Exit main()");
    }

    public static void methodA() {
        System.out.println("Enter methodA()");
        methodB();
        System.out.println("Exit methodA()");
    }

    public static void methodB() {
        System.out.println("Enter methodB()");
        methodC();
        System.out.println("Exit methodB()");
    }

    public static void methodC() {
        System.out.println("Enter methodC()");
        System.out.println("Exit methodC()");
    }
}
```

```
Enter main()

Enter methodA()

Enter methodB()

Enter methodC()

Exit methodC()

Exit methodB()

Exit methodA()

Exit main()
```

Como se observa en la salida, la secuencia de eventos es:

1. JVM invoca a **main()**.
2. **main()** se coloca en la pila de llamadas, antes de invocar a **methodA()**.
3. **methodA()** se coloca en la pila de llamadas, antes de invocar a **methodB()**.
4. **methodB()** se coloca en la pila de llamadas, antes de invocar a **methodC()**.
5. **methodC()** se completa.
6. **methodB()** se retira de la pila de llamadas y se completa.
7. **methodA()** se retira de la pila de llamadas y se completa.
8. **main()** se retira de la pila de llamadas y se completa. El programa termina.



Supongamos que modificamos **methodC()** para realizar una operación de "división por 0", lo que desencadena una **ArithmeticException**:

```
public static void methodC() {
    System.out.println("Enter methodC()");
    System.out.println(1 / 0);  // divide-by-0 triggers an ArithmeticException
    System.out.println("Exit methodC()");
}
```

El mensaje de la excepción muestra claramente el **rastreo de la pila de llamadas a métodos** con los números de línea de las sentencias relevantes:

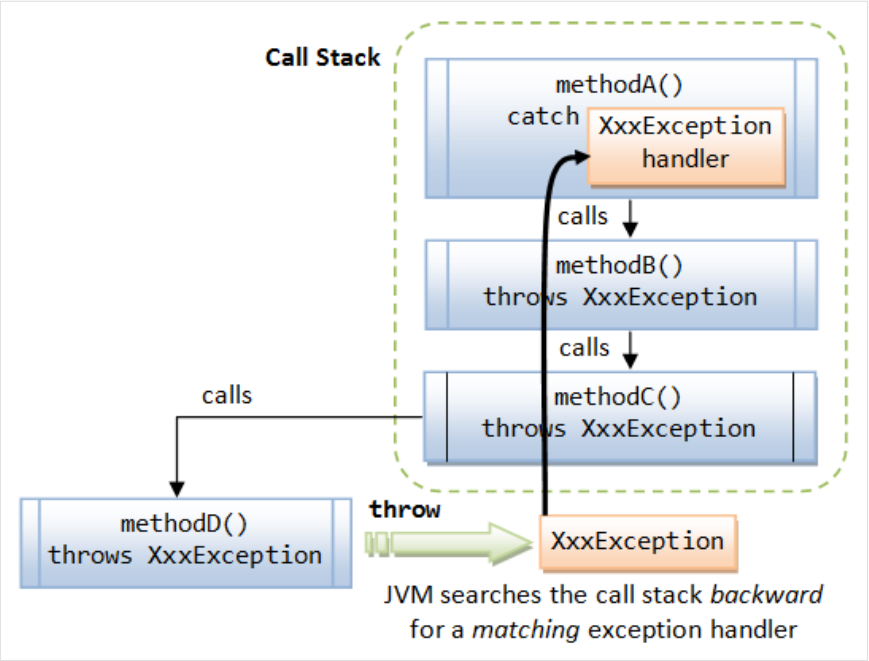
```
Enter main()
Enter methodA()
Enter methodB()
Enter methodC()
Exception in thread "main" java.lang.ArithmeticException: / by zero
          at MethodCallStackDemo.methodC(MethodCallStackDemo.java:22)
          at MethodCallStackDemo.methodB(MethodCallStackDemo.java:16)
          at MethodCallStackDemo.methodA(MethodCallStackDemo.java:10)
          at MethodCallStackDemo.main(MethodCallStackDemo.java:4)
```

MethodC() desencadena una **ArithmeticException**. Como no maneja esta excepción, se retira inmediatamente de la pila de llamadas. **MethodB()** tampoco maneja esta excepción y se retira de la pila de llamadas. Lo mismo ocurre con **methodA()** y el método **main()**. El método **main()** pasa de nuevo a la JVM, la cual termina abruptamente el programa e imprime el rastreo de la pila de llamadas, como se muestra.

1.3 Excepciones y Pilas de Llamadas

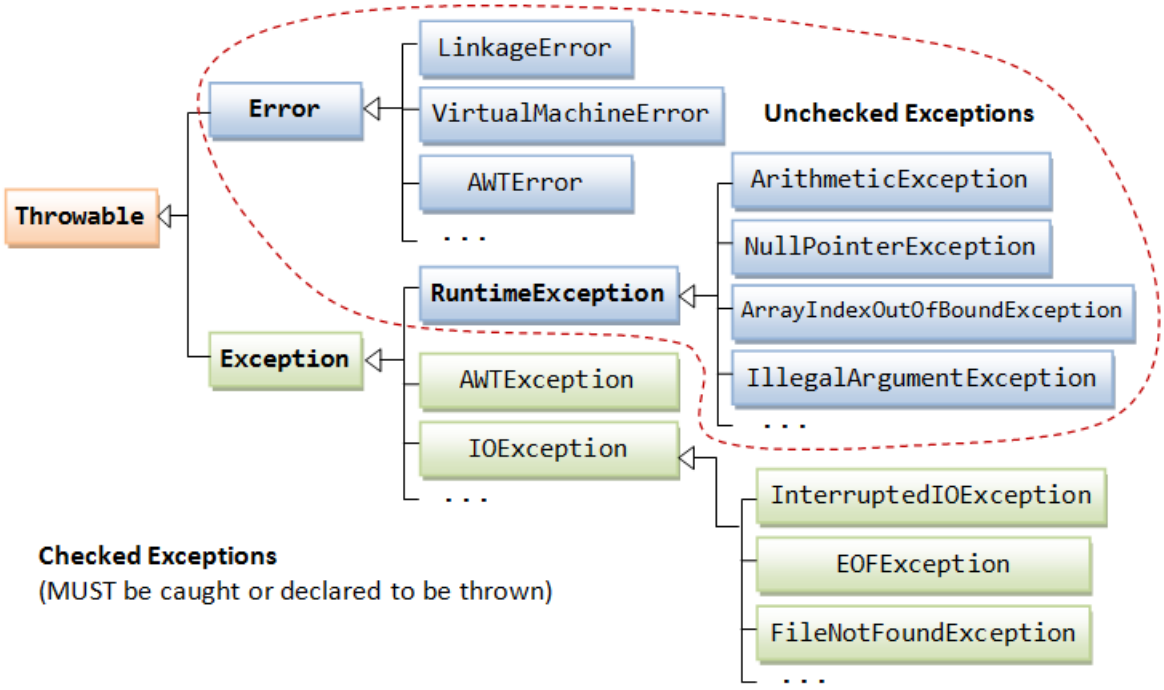
Cuando ocurre una excepción dentro de un método en Java, el método crea un objeto **Exception** y pasa el objeto **Exception** a la JVM (en términos de Java, el método "lanza" una **Exception**). El objeto **Exception** contiene el tipo de la excepción y el estado del programa cuando ocurre la excepción. La JVM es responsable de encontrar un **manejador de excepciones** para procesar el objeto **Exception**. Busca hacia atrás a través de la pila de llamadas hasta que encuentra un manejador de excepciones que coincida con esa clase particular de objeto **Exception** (en términos de Java, se llama "atrapar" la **Exception**). Si la JVM no encuentra un manejador de excepciones coincidente en todos los métodos de la pila de llamadas, termina el programa.

Este proceso se ilustra de la siguiente manera. Supongamos que **methodD()** encuentra una condición anormal y lanza una **XxxException** a la JVM. La JVM busca hacia atrás a través de la pila de llamadas por un manejador de excepciones coincidente. Encuentra que **methodA()** tiene un manejador para **XxxException** y pasa el objeto de excepción al manejador. Nótese que **methodC()** y **methodB()** están obligados a declarar "throws XxxException" en sus firmas de método para poder compilar el programa.



1.4 Definición de Clases de Excepciones: - Throwable, Error, Exception & RuntimeException

La figura a continuación muestra la jerarquía de las clases **Exception**. La clase base para todos los objetos **Exception** es **java.lang.Throwable**, junto con sus dos subclases **java.lang.Exception** y **java.lang.Error**.



La clase **Error** describe errores internos del sistema (por ejemplo, **VirtualMachineError**, **LinkageError**) que raramente ocurren. Si tal error ocurre, hay poco que puedas hacer y el programa finalizará abruptamente.

La clase **Exception** describe los errores causados por tu programa (por ejemplo, **FileNotFoundException**, **IOException**). Estos errores podrían ser capturados y manejados por tu programa (por ejemplo, realizar una acción alternativa o hacer una salida elegante cerrando todos los archivos, conexiones de red y de base de datos).

Como se ilustra, las subclases de **Error** y **RuntimeException** son conocidas como **excepciones no verificadas**. Estas excepciones no son comprobadas por el compilador, y por lo tanto, no necesitan ser capturadas o declaradas como lanzadas en tu programa. Esto se debe a que no hay mucho que puedas hacer con estas excepciones. Por ejemplo, una "división por 0" desencadena una **ArithmeticException**, un índice de arreglo fuera de límites desencadena una **ArrayIndexOutOfBoundsException**, las cuales son realmente errores lógicos de programación que deberían haberse corregido en tiempo de compilación, en lugar de dejarlo al manejo de excepciones en tiempo de ejecución.

Todas las otras excepciones se llaman **excepciones verificadas**. Son comprobadas por el compilador y deben ser capturadas o declaradas como lanzadas.

1.5 Operaciones de manejo de excepciones

Se utilizan cinco palabras clave en el manejo de excepciones: **try**, **catch**, **finally**, **throws** y **throw** (toma en cuenta que hay una diferencia entre **throw** y **throws**).

El manejo de excepciones en Java consta de tres operaciones:
1. Declarar excepciones.
2. Lanzar una excepción.
3. Atrapar una excepción.

Declarando Excepciones:

Un método en Java debe declarar en su firma los tipos de excepción verificada que puede "lanzar" desde su cuerpo, mediante la palabra clave "throws".

Por ejemplo, supongamos que **methodD()** se define de la siguiente manera:

```
public void methodD() throws XxxException, YyyException {
    // method body throw XxxException and YyyException
}
```

La firma del método indica que ejecutar **methodD()** puede encontrar dos excepciones verificadas: **XxxException** y **YyyException**. En otras palabras, algunas de las condiciones anormales dentro de **methodD()** pueden desencadenar **XxxException** o **YyyException**.
Las excepciones que pertenecen a **Error**, **RuntimeException** y sus **subclases** no necesitan ser declaradas. Estas excepciones se llaman **excepciones no verificadas** porque no son comprobadas por el compilador.

Lanzando una excepción

Cuando una operación en Java encuentra una situación anormal, el método que contiene la sentencia errónea debe crear un objeto **Exception** apropiado y lanzarlo al entorno de ejecución de Java mediante la sentencia "throw XxxException". Por ejemplo,

```
public void methodD() throws XxxException, YyyException {   // method's signature
    // method's body
    ...
    ...
    // XxxException occurs
    if ( ... )
        throw new XxxException(...);   // construct an XxxException object and throw to JVM
    ...
    // YyyException occurs
    if ( ... )
        throw new YyyException(...);   // construct an YyyException object and throw to JVM
    ...
}
```

Ten en cuenta que la palabra clave para declarar una excepción en la firma del método es "throws" y la palabra clave para lanzar un objeto de excepción dentro del cuerpo del método es "throw".

Capturando una Excepción

Cuando un método lanza una excepción, la JVM busca hacia atrás a través de la pila de llamadas un manejador de excepciones que coincida. Cada manejador de excepciones puede manejar una clase particular de excepción. Un manejador de excepciones que maneja una clase específica también puede manejar sus subclases. Si no se encuentra un manejador de excepciones en la pila de llamadas, el programa termina.

Por ejemplo, supongamos que **methodD()** declara que puede lanzar **XxxException** y **YyyException** en su firma, como sigue:

```
public void methodD() throws XxxException, YyyException { ...... }
```

Para usar **methodD()** en tu programa (digamos en **methodC()**), puedes:

1. Envolver la llamada a **methodD()** dentro de un try-catch (o try-catch-finally) de la siguiente manera. Cada bloque catch puede contener un manejador de excepciones para un tipo de excepción.

```
public void methodC() {  // no exception declared
    ......
    try {
        ......
        // uses methodD() which declares XxxException & YyyException
        methodD();
        ......
    } catch (XxxException ex) {
        // Exception handler for XxxException
        ......
    } catch (YyyException ex} {
        // Exception handler for YyyException
        ......
    } finally {   // optional
        // These codes always run, used for cleaning up
        ......
    }
    ......
}
```

2. Suponer que **methodC()**, que llama a **methodD()**, no desea manejar las excepciones (a través de un try-catch), puede declarar que estas excepciones se lancen hacia arriba en la pila de llamadas en su firma de la siguiente manera:

```
public void methodC() throws XxxException, YyyException {    // for next higher-level method to
handle
    ...
    // uses methodD() which declares "throws XxxException, YyyException"
    methodD();   // no need for try-catch
    ...
}
```

En este caso, si una **XxxException** o **YyyException** es lanzada por **methodD()**, la JVM terminará **methodD()** así como **methodC()** y pasará el objeto de excepción hacia arriba en la pila de llamadas al llamador de **methodC()**.

1.6 try-catch-finally

La sintaxis de try-catch-finally es:

```
try {
    // main logic, uses methods that may throw Exceptions
    ......
} catch (Exception1 ex) {
    // error handler for Exception1
    ......
} catch (Exception2 ex) {
    // error handler for Exception1
    ......
} finally {   // finally is optional
    // clean up codes, always executed regardless of exceptions
    ......
}
```

Si no ocurre ninguna excepción durante la ejecución del bloque try, todos los bloques catch se omiten, y el bloque finally se ejecutará después del bloque try. Si una de las sentencias en el bloque try lanza una excepción, el entorno de ejecución de Java ignora el resto de las sentencias en el bloque try y comienza a buscar un manejador de excepciones coincidente. Hace coincidir el tipo de excepción con cada uno de los bloques catch secuencialmente. Si un bloque catch atrapa esa clase de excepción o atrapa una **superclase** de esa excepción, se ejecutarán las sentencias en ese bloque catch. Las sentencias en el bloque finally se ejecutan entonces después de ese bloque catch. El programa continúa en la siguiente sentencia después del try-catch-finally, a menos que sea terminado prematuramente o se salga del flujo normal.

Si ninguno de los bloques catch coincide, la excepción se pasará hacia arriba en la pila de llamadas. El método actual ejecuta la cláusula **finally** (si hay alguna) y se retira de la pila de llamadas. El llamador sigue los mismos procedimientos para manejar la excepción.

El bloque **finally** se ejecutará en todos los casos, independientemente de si ocurre o no una excepción (a menos que la JVM encuentre un error grave o se llame a **System.exit()** en el bloque catch).

Ejemplo 1

```
import java.util.Scanner;
import java.io.File;
import java.io.FileNotFoundException;
public class TryCatchFinally {
    public static void main(String[] args) {
        try {         // main logic
            System.out.println("Start of the main logic");
            System.out.println("Try opening a file ...");
            Scanner in = new Scanner(new File("test.in"));
            System.out.println("File Found, processing the file ...");
            System.out.println("End of the main logic");
        } catch (FileNotFoundException ex) {    // error handling separated from the main logic
            System.out.println("File Not Found caught ...");
        } finally {   // always run regardless of exception status
            System.out.println("finally-block runs regardless of the state of exception");
        }
        // after the try-catch-finally
        System.out.println("After try-catch-finally, life goes on...");
    }
}
```

Esta es la salida cuando se lanza la exepción `FileNotFoundException`:

```
Start of the main logic
```

```
Try opening a file ...

File Not Found caught ...

finally-block runs regardless of the state of exception

After try-catch-finally, life goes on...
```

Esta es la salida cuando no se lanzan excepciones:

```
Start of the main logic

Try opening a file ...

File Found, processing the file ...

End of the main logic

finally-block runs regardless of the state of exception

After try-catch-finally, life goes on...
```

Ejemplo 2

```
1
2
3
4
5
6
7
8
9
1
0
1
1
1
2
1
3
1
4
1
5
1
6
1
7
1
8
1
9
2
0
public class MethodCallStackDemo {
    public static void main(String[] args) {
        System.out.println("Enter main()");
        methodA();
        System.out.println("Exit main()");
    }

    public static void methodA() {
        System.out.println("Enter methodA()");
        try {
            System.out.println(1 / 0);
                // A divide-by-0 triggers an ArithmeticException - an unchecked
exception
                // This method does not catch ArithmeticException
                // It runs the "finally" and popped off the call stack
        } finally {
            System.out.println("finally in methodA()");
        }
        System.out.println("Exit methodA()");
    }
}
```

```
Enter main()

Enter methodA()

finally in methodA()

Exception in thread "main" java.lang.ArithmeticException: / by zero

          at MethodCallStackDemo.methodA(MethodCallStackDemo.java:11)

          at MethodCallStackDemo.main(MethodCallStackDemo.java:4)
```

try-catch-finally

- Un bloque try debe ir acompañado por al menos un bloque catch o un bloque finally.
- Puedes tener múltiples bloques catch. Cada bloque catch atrapa solo un tipo de excepción.
- Un bloque catch requiere un argumento, que es un objeto **throwable** (es decir, una subclase de **java.lang.Throwable**), como sigue:

```
catch (AThrowableSubClass aThrowableObject) {
    // exception handling codes
}
```

- Puedes usar los siguientes métodos para recuperar el tipo de la excepción y el estado del programa del objeto **Throwable**:

printStackTrace(): Imprime este **Throwable** y su rastreo de pila de llamadas en el flujo de error estándar **System.err**. La primera línea de las salidas contiene el resultado de **toString()**, y las líneas restantes son el rastreo de pila. Este es el manejador más común, si no hay algo mejor que puedas hacer. Por ejemplo:

```
try {
    Scanner in = new Scanner(new File("test.in"));
    // process the file here
    ......
} catch (FileNotFoundException ex) {
    ex.printStackTrace();
}
```

También puedes usar **printStackTrace(PrintStream s)** o **printStackTrace(PrintWriter s)**.
getMessage(): Devuelve el mensaje especificado si el objeto se construye utilizando el constructor **Throwable(String message)**.
toString(): Devuelve una descripción corta de este objeto **Throwable**, que consiste en el nombre de la clase, dos puntos ':', y un mensaje de **getMessage()**.

Un bloque catch que atrapa una clase de excepción específica también puede atrapar sus **subclases**. Por lo tanto, **catch(Exception ex) {...}** atrapa todo tipo de excepciones. Sin embargo, esta no es una buena práctica ya que el manejador de excepciones que es demasiado general puede atrapar accidentalmente algunas excepciones de subclases que no pretende.
El orden de los bloques catch es importante. Una subclase debe ser atrapada (y colocada al frente) antes que su superclase. De lo contrario, recibirás un error de compilación "la excepción **XxxException** ya ha sido atrapada".
El bloque **finally** está destinado a código de limpieza como cerrar el archivo, conexión a la base de datos, independientemente de si el bloque try tiene éxito. El bloque finally siempre se ejecuta (a menos que el bloque catch termine prematuramente el método actual).

¿Qué pasa si no te importan las excepciones?

Esto es una mala práctica profesional: siempre deben imporante las excepciones y su tratamiento.

Sobreescritura y sobrecarga de métodos

Un método sobrescrito debe tener la misma lista de argumentos y tipo de retorno (o subclase de su original desde JDK 1.5). Un método sobrecargado debe tener una lista de argumentos diferente, pero puede tener cualquier tipo de retorno.

Un método sobrescrito no puede tener un acceso más restringido. Por ejemplo, un método con acceso **protected** puede ser sobrescrito para tener acceso protected o public, pero no **private** o acceso por defecto. Esto se debe a que un método sobrescrito se considera un reemplazo de su original, por lo tanto, no puede ser más restrictivo.

Un método sobrescrito no puede declarar tipos de excepción que no fueron declarados en su original. Sin embargo, puede declarar tipos de excepción que son iguales o subclase de su original. No necesita declarar todas las excepciones como su original. Puede lanzar menos excepciones que el original, pero no más.

Un método sobrecargado debe diferenciarse por su lista de argumentos. No puede diferenciarse por el tipo de retorno, las excepciones y el modificador, lo cual es ilegal. Puede tener cualquier tipo de retorno, modificador de acceso y excepciones, siempre que se pueda diferenciar por la lista de argumentos.

1.7Clases de Excepción usadas comúnmente

`ArrayIndexOutOfBoundsException`: Lanzada por la JVM cuando intentas acceder a una posición de un array que está fuera del rango creado, como por ejemplo:

```
int[] anArray = new int[3];

System.out.println(anArray[3]);
```

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 3
```

`NullPointerException`: Lanzada por la JVM cuando nuestro código intenta usar una referencia a un objeto que en realidad es nula, como por ejemplo:

```
String[] strs = new String[3];

System.out.println(strs[0].length());
```

```
Exception in thread "main" java.lang.NullPointerException
```

`NumberFormatException`: Se lanza a nivel de programa (p.ej. por `Integer.parseInt()`) cuando se intenta convertir una cadena a un número, pero la cadena no tiene el formato apropiado. Por ejemplo:

```
Integer.parseInt("abc");
```

```
Exception in thread "main" java.lang.NumberFormatException: For input string: "abc"
```

`ClassCastException`: Lanzada por la JVM cuando se intenta cambiar el tipo de un objeto a otro, pero no es posible por no ser compatibles. Por ejemplo,

```
Object o = new Object();

Integer i = (Integer)o;
```

```
Exception in thread "main" java.lang.ClassCastException: java.lang.Object cannot be cast to
java.lang.Integer
```

`IllegalArgumentException`: Lanzada a nivel de programación para indicar que un método ha recibido argumentos no apropiados. Este tipo de excepción puede usarse en nuestros propios métodos para lanzarla cuando recibimos parámetros con valores que hacen que nuestro método falle.

`IllegalStateException`: se lanza programáticamente cuando se invoca un método y el programa no se encuentra en un estado apropiado para que ese método realice su tarea. Esto ocurre cuando un método se invoca fuera de secuencia, o quizás un método solo se permite invocar una vez y se intenta invocarlo nuevamente.

`NoClassDefFoundError`: Lanzada por la JVM cuando se solicita una clase que no puede ser encontrada (no se ha programado, o se ha situado en una carpeta distinta a la esperada, ya sea en la del programa, en las bibliotecas o en el classpath).

1.8 Creando nuestras propias clases de Excepciones

Deberías intentar reutilizar las clases **Exception** proporcionadas en el JDK, por ejemplo, **IndexOutOfBoundsException**, **ArithmeticException**, **IOException** y **IllegalArgumentException**. Pero siempre puedes crear tus propias clases **Exception** extendiéndolas de la clase **Exception** o una de sus subclases.

Ten en cuenta que **RuntimeException** y sus subclases no son verificadas por el compilador y no necesitan ser declaradas en la firma del método. Por lo tanto, úsalas con cuidado, ya que no serás informado y puede que no estés consciente de las excepciones que pueden ocurrir al usar ese método (y por lo tanto no tener los códigos de manejo de excepciones adecuados) – una mala práctica de ingeniería de software.

Ejemplo

```
// Create our own exception class by subclassing Exception. This is a checked exception
public class MyMagicException extends Exception {
    public MyMagicException(String message) {  //constructor
        super(message);
    }
}
```

```
public class MyMagicExceptionTest {
    // This method "throw MyMagicException" in its body.
    // MyMagicException is checked and need to be declared in the method's signature
    public static void magic(int number) throws MyMagicException {
        if (number == 8) {
            throw (new MyMagicException("you hit the magic number"));
        }
        System.out.println("hello");  // skip if exception triggered
    }

    public static void main(String[] args) {
        try {
            magic(9);   // does not trigger exception
            magic(8);   // trigger exception
        } catch (MyMagicException ex) {   // exception handler
            ex.printStackTrace();
        }
    }
}
```

La salida del ejemplo sería:

```
hello

MyMagicException: you hit the magic number

          at MyMagicExceptionTest.magic(MyMagicExceptionTest.java:6)

          at MyMagicExceptionTest.main(MyMagicExceptionTest.java:14)
```